

# Recursive Semantic-to-Geometric Decomposition via Gaussian Splatting

Nikita Gorshkov  
Radiance Labs

## Abstract

Current text-to-3D generation methods produce objects as monolithic geometric outputs, lacking explicit compositional structure. We identify a fundamental gap: no existing method recursively decomposes natural language descriptions into hierarchical composition trees terminating at individual geometric primitives. We propose Recursive Semantic-to-Geometric Decomposition (RSGD), an architecture that pairs a small language model (the Decomposer) with a Gaussian splatting renderer to produce 3D scenes through recursive expansion of semantic concepts into sub-concepts, ultimately grounding at individual 3D Gaussian primitives. Each intermediate node in the composition tree corresponds to a “blob,” a probability distribution over possible decompositions that captures structural variation (e.g., a Disney castle versus a motte-and-bailey both satisfy “castle” but differ in decomposition topology). GloVe word embeddings provide the addressing mechanism for word-level discrimination at each recursive step, resolving a subword tokenization barrier we empirically identify in flat routing architectures. We present evidence from three generations of flat-routing experiments (Raum 1.0, 1.1, 1.2) demonstrating that flat classification ceilings at 0.1% blob accuracy when subword collisions span 35 collision groups across 300 shape classes. Recursive decomposition sidesteps this barrier by leveraging the language model for compositional structure and GloVe for per-word identity, each operating in its regime of competence.

## 1 Introduction

The text-to-3D generation pipeline has achieved remarkable progress: given a natural language prompt, systems such as DreamFusion [16], GaussianDreamer [23], and GSGEN [4] produce plausible 3D assets via score distillation sampling or feed-forward generation. Yet these approaches treat the generated object as an opaque geometric artifact. The internal structure of a castle, the fact that it comprises towers, walls, a gate, and a keep, is not represented in the output. The geometry is entangled; editing, recomposition, and structural reasoning become post-hoc reconstruction problems.

Compositional scene generation methods (GALA3D [26], SceneWiz3D [25]) address multi-object layout but stop at the object boundary. They compose objects, not sub-parts. Part-based shape generation (PartNeRF [20], BSP-Net [3]) decomposes geometry into parts, but does not connect the decomposition to natural language semantics.

We observe a missing link: **recursive semantic decomposition all the way to individual geometric primitives**. If “castle” decomposes into {tower, gate, keep, wall}, and “tower” further decomposes into {cylinder\_body, cone\_roof, flag}, then eventually every leaf node corresponds to a single 3D Gaussian splat with concrete parameters: position  $\mu \in \mathbb{R}^3$ , covariance  $\Sigma \in \mathbb{R}^{3 \times 3}$ , opacity  $\alpha \in [0, 1]$ , and color features  $\mathbf{f} \in \mathbb{R}^C$ . The entire scene becomes a tree whose leaves are Gaussians and whose internal nodes carry compositional semantics.

This paper makes three contributions:

1. **Gap identification.** We demonstrate empirically that flat routing architectures (mapping word embeddings directly to shape classes) hit a hard ceiling imposed by subword tokenization collisions, and formalize the resulting bottleneck.
2. **Architecture proposal.** We introduce Recursive Semantic-to-Geometric Decomposition (RSGD), which uses a trained Decomposer language model to produce composition trees, blob distributions for structural variation, and GloVe-addressed terminal Gaussians.
3. **Empirical grounding.** We report results from three generations of flat routing experiments (Raum 1.0 through 1.2) that motivate the recursive approach, including quantitative analysis of the subword collision barrier.

## 2 Related Work

### 2.1 Text-to-3D Generation

DreamFusion [16] introduced Score Distillation Sampling (SDS) to lift 2D diffusion priors into 3D by optimizing a NeRF representation against a frozen text-to-image model. Magic3D [13] improved resolution via a coarse-to-fine strategy with DMTet meshes. ProlificDreamer [22] proposed Variational Score Distillation for higher-fidelity outputs.

The Gaussian splatting family, building on 3D Gaussian Splatting [9], brought explicit point-based representations into text-to-3D. GaussianDreamer [23] combines 3D Gaussian initialization from point-cloud diffusion with SDS refinement. GSGEN [4] directly optimizes Gaussians under SDS guidance. LucidDreamer [12] addresses the Janus problem via interval score matching.

All of these methods produce monolithic outputs: the geometry has no explicit compositional structure. One cannot ask “which Gaussians form the tower?” without post-hoc segmentation. Our work produces structured outputs by construction.

### 2.2 Compositional Scene Generation

GALA3D [26] leverages a large language model to plan multi-object layouts, generating individual objects and composing them spatially. SceneWiz3D [25] performs scene-level generation with object-level disentanglement. Set-the-Scene [5] arranges generated objects according to spatial constraints.

These approaches compose at the object granularity. “A castle on a hill” might decompose into two objects (castle, hill), but the internal structure of the castle is generated monolithically. RSGD extends composition recursively below the object boundary to the primitive level.

### 2.3 Part-based Shape Generation

PartNeRF [20] represents shapes as compositions of local NeRF parts, learning to decompose objects into semantically meaningful components. BSP-Net [3] decomposes shapes into convex parts via binary space partitions. Neural Parts [15] learn implicit primitive decompositions supervised by part annotations.

These methods discover geometric parts but lack linguistic grounding. The parts are not named, and the decomposition is not driven by natural language semantics. RSGD bridges the gap: decomposition is linguistically structured (each node has a semantic label) and proceeds recursively from the language domain into geometry.

## 2.4 Language-Guided Planning

SayCan [1] uses language models to decompose high-level instructions into executable robot actions, grounding language in affordances. Code-as-Policies [11] generates executable code from language for robot control. ProgPrompt [19] uses programmatic prompting for task planning.

These works demonstrate that language models can serve as hierarchical planners, decomposing abstract goals into concrete steps. RSGD applies the same principle to 3D scene construction: the language model decomposes a scene description into a composition tree, with the “actions” being geometric primitive placements rather than robot commands.

## 2.5 Hierarchical Representations

VQ-VAE [21] and its hierarchical extensions (VQ-VAE-2 [18]) learn discrete multi-scale codebooks. Residual Quantization [10] stacks quantizers for progressive refinement. In 3D, hierarchical codebooks have been applied to point clouds and voxel grids.

RSGD’s blob distributions share the spirit of discrete codebooks (each blob is a distribution over possible compositions), but differ in that the hierarchy is semantically labeled and the tree structure varies per input rather than being fixed across all inputs.

# 3 Background: Semantic Gaussian Splatting

We build on the Semantic Gaussian Splatting (SGS) framework [6], which extends 3D Gaussian Splatting with semantic embeddings and multi-pass rendering.

## 3.1 The Gaussian Primitive

A 3D Gaussian primitive  $g$  is defined by:

$$g = (\boldsymbol{\mu}, \boldsymbol{\Sigma}, \alpha, \mathbf{f}) \quad (1)$$

where  $\boldsymbol{\mu} \in \mathbb{R}^3$  is the mean position,  $\boldsymbol{\Sigma} \in \mathbb{R}^{3 \times 3}$  is the covariance matrix (parameterized as  $\boldsymbol{\Sigma} = \mathbf{R}\mathbf{S}\mathbf{S}^T\mathbf{R}^T$  with rotation  $\mathbf{R}$  and scale  $\mathbf{S}$ ),  $\alpha \in [0, 1]$  is opacity, and  $\mathbf{f} \in \mathbb{R}^C$  encodes appearance (spherical harmonics or learned features).

In SGS, each Gaussian additionally carries a semantic embedding  $\mathbf{e} \in \mathbb{R}^D$  that links it to the linguistic domain. This embedding enables querying, editing, and compositional reasoning over the scene at the primitive level.

## 3.2 Transmittance-Weighted Rendering

For a pixel with ray  $\mathbf{r}$ , the rendered color is:

$$C(\mathbf{r}) = \sum_{i=1}^N T_i \alpha_i \mathbf{c}_i \quad (2)$$

where  $T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$  is the accumulated transmittance,  $\alpha_i$  is the contribution of the  $i$ -th Gaussian (computed from its projected opacity and the ray-Gaussian distance), and  $\mathbf{c}_i$  is the color of the  $i$ -th Gaussian evaluated via spherical harmonics or feature decoding.

Gaussians are sorted front-to-back along the ray, and alpha-compositing proceeds sequentially. This formulation is differentiable with respect to all Gaussian parameters, enabling gradient-based optimization.

### 3.3 Multi-Pass Refinement

SGS employs a multi-pass rendering strategy:

1. **Coarse pass:** Low-resolution rendering with a subset of Gaussians for initial layout.
2. **Refinement pass:** Full-resolution rendering with adaptive density control (splitting and pruning).
3. **Semantic pass:** Rendering semantic embeddings for linguistic grounding.

Each pass shares the same transmittance-weighted compositing but operates at different resolutions or over different subsets of the primitive set. This multi-pass design enables progressive scene construction, which RSGD leverages for top-down recursive expansion.

## 4 Method: Recursive Decomposition

### 4.1 Architecture Overview

The architecture operates as follows. Given a text input, GloVe embeddings provide word-level identity to the Decomposer, which produces a composition tree. Each internal node represents a semantic concept; each edge encodes a spatial relation (relative position, scale, orientation). Leaf nodes are terminal primitives, each mapped to a concrete Gaussian via the Terminal Mapper. Between the tree structure and the terminal mapping, a Blob Sampler introduces structural variation by sampling from learned distributions over decomposition topologies.

### 4.2 The Decomposer (Planck LM)

The Decomposer is a 100M-parameter transformer language model (Planck) trained on Wikipedia text to acquire compositional world knowledge. Given a concept embedding, it produces a set of child concepts with spatial relations:

$$\text{Decompose}(\mathbf{z}_{\text{parent}}) \rightarrow \{(\mathbf{z}_{\text{child}_i}, \mathbf{r}_i)\}_{i=1}^k \quad (3)$$

where  $\mathbf{z}_{\text{parent}} \in \mathbb{R}^D$  is the parent concept embedding,  $\mathbf{z}_{\text{child}_i}$  is the child concept embedding,  $\mathbf{r}_i = (\Delta\boldsymbol{\mu}_i, \Delta\mathbf{s}_i, \Delta\boldsymbol{\theta}_i)$  encodes the relative spatial transform (translation, scale, rotation), and  $k$  is the branching factor (variable per node).

The Decomposer is not a standard autoregressive text generator. It operates in embedding space, taking a  $D$ -dimensional vector and producing a set of child vectors with spatial offsets. This is implemented as:

1. The parent embedding  $\mathbf{z}_{\text{parent}}$  is projected into the transformer’s latent space.
2. A cross-attention mechanism attends over a learned “part vocabulary” to select relevant children.
3. A spatial head predicts relative transforms for each selected child.
4. A termination head predicts whether each child is terminal (leaf) or requires further decomposition.

The branching factor  $k$  is determined by a learned halting mechanism: children are generated sequentially, and generation stops when the termination probability exceeds a threshold  $\tau$ .

### 4.3 Blob Distributions

A key insight is that “castle” does not have a single correct decomposition. A Disney fairy-tale castle and a medieval motte-and-bailey castle share the label “castle” but differ radically in structure. We model this via **blobs**: probability distributions over composition tree topologies.

**Definition 1** (Blob Distribution). *A blob  $\mathcal{B}$  for concept  $c$  is a distribution over composition trees:*

$$\mathcal{B}_c = p(\mathcal{T} | c) \quad (4)$$

where  $\mathcal{T}$  is a composition tree.

In practice, we parameterize this as a latent variable model:

$$p(\mathcal{T} | c) = \int p(\mathcal{T} | \mathbf{z}) p(\mathbf{z} | c) d\mathbf{z} \quad (5)$$

where  $\mathbf{z} \in \mathbb{R}^L$  is a latent code sampled from a learned prior conditioned on the concept embedding. Different samples of  $\mathbf{z}$  produce different tree topologies (different branching factors, different child sets, different spatial arrangements), capturing the structural diversity within a single concept.

The blob latent space is trained via a variational objective:

$$\mathcal{L}_{\text{blob}} = \mathbb{E}_{q(\mathbf{z} | \mathcal{T}, c)} [\log p(\mathcal{T} | \mathbf{z})] - \beta \text{KL}[q(\mathbf{z} | \mathcal{T}, c) \parallel p(\mathbf{z} | c)] \quad (6)$$

This encourages the latent space to be smooth (nearby  $\mathbf{z}$  values produce similar trees) while being expressive enough to capture the full range of structural variation.

### 4.4 Terminal Primitive Renderer

At the leaves of the composition tree, terminal nodes must be converted to concrete Gaussian parameters. The Terminal Mapper takes a leaf embedding  $\mathbf{z}_{\text{leaf}}$  and its accumulated spatial context (the product of relative transforms along the path from root to leaf) and outputs Gaussian parameters:

$$(\boldsymbol{\mu}, \boldsymbol{\Sigma}, \alpha, \mathbf{f}) = \text{TerminalMap}(\mathbf{z}_{\text{leaf}}, \mathbf{T}_{\text{accumulated}}) \quad (7)$$

The accumulated transform  $\mathbf{T}_{\text{accumulated}}$  is computed by composing all relative transforms along the root-to-leaf path:

$$\mathbf{T}_{\text{accumulated}} = \mathbf{T}_{\text{root}} \circ \mathbf{T}_1 \circ \mathbf{T}_2 \circ \dots \circ \mathbf{T}_d \quad (8)$$

where  $d$  is the depth of the leaf. This hierarchical composition of transforms ensures that local part placements are defined relative to their parent, enabling structural coherence.

Critically, the leaf embedding  $\mathbf{z}_{\text{leaf}}$  is addressed via GloVe. Because GloVe embeddings provide unique, well-separated vectors for each word in the vocabulary, they serve as reliable “addresses” for terminal primitives. This resolves the subword tokenization problem (Section 5.2): the terminal mapper never sees subword tokens, only whole-word GloVe vectors.

### 4.5 Training: Supervising Decomposition Trees

Training RSGD requires supervision for the composition tree structure. We propose three complementary supervision signals:

**Render loss.** The final rendered image is compared against ground-truth multi-view images via a photometric loss:

$$\mathcal{L}_{\text{render}} = \sum_v \|C_v - \hat{C}_v\|_1 + \lambda_{\text{SSIM}}(1 - \text{SSIM}(C_v, \hat{C}_v)) \quad (9)$$

This provides end-to-end gradient flow from pixels back through the renderer, terminal mapper, and (via the reparameterization trick for blob sampling) into the Decomposer.

**Tree structure loss.** When ground-truth part annotations are available (e.g., from PartNet [14] or ShapeNet parts), we supervise the tree topology directly:

$$\mathcal{L}_{\text{tree}} = \text{TreeEditDistance}(\mathcal{T}_{\text{pred}}, \mathcal{T}_{\text{gt}}) \quad (10)$$

computed as a differentiable relaxation of tree edit distance using soft attention over predicted and ground-truth nodes.

**Semantic consistency loss.** Each internal node’s embedding should be semantically consistent with its children. We enforce this via a contrastive loss:

$$\mathcal{L}_{\text{sem}} = -\log \frac{\exp(\text{sim}(\mathbf{z}_{\text{parent}}, \bar{\mathbf{z}}_{\text{children}})/\tau)}{\sum_j \exp(\text{sim}(\mathbf{z}_{\text{parent}}, \bar{\mathbf{z}}_j)/\tau)} \quad (11)$$

where  $\bar{\mathbf{z}}_{\text{children}}$  is the mean of child embeddings and the denominator sums over negative examples (children of other parents in the batch).

The total training objective is:

$$\mathcal{L} = \mathcal{L}_{\text{render}} + \lambda_t \mathcal{L}_{\text{tree}} + \lambda_s \mathcal{L}_{\text{sem}} + \lambda_b \mathcal{L}_{\text{blob}} \quad (12)$$

## 4.6 Inference: Recursive Expansion with Early Stopping

At inference time, the Decomposer expands the tree top-down. The procedure is described in Algorithm 1.

---

### Algorithm 1: Recursive Tree Expansion

---

**Input:** node (with embedding and transform), depth,  $d_{\text{max}}$   
**Output:** List of Gaussian primitives  
**Function** `Expand(node, depth):`  
  **if** `depth > dmax` **or** `Terminal(node)` **then**  
    **return** `TerminalMap(node.embedding, node.transform);`  
  `children`  $\leftarrow$  `Decompose(node.embedding);`  
  `gaussians`  $\leftarrow$  `[];`  
  **foreach** `child` **in** `children` **do**  
    `child.transform`  $\leftarrow$  `node.transform`  $\circ$  `child.relative_transform;`  
    `gaussians.extend(Expand(child, depth + 1));`  
  **return** `gaussians;`

---

Early stopping is controlled by two mechanisms:

1. **Termination head:** The Decomposer’s termination head predicts  $p(\text{terminal}|\mathbf{z})$ . When this exceeds threshold  $\tau$ , expansion halts.
2. **Depth budget:** A hard maximum depth  $d_{\max}$  prevents unbounded recursion. In practice,  $d_{\max} = 6$  suffices for most scenes (yielding up to  $\sim k^6$  Gaussians for branching factor  $k$ ).

The number of Gaussians in the final scene is determined by the tree’s leaf count, which varies per input. This adaptive complexity is a feature: simple concepts (“sphere”) terminate quickly with few Gaussians, while complex concepts (“castle”) expand deeply.

## 5 Empirical Motivation

This section reports results from three generations of routing experiments within the SGS project. These experiments motivated the recursive architecture by demonstrating the ceiling of flat approaches.

### 5.1 Flat Routing Bridges and Their Ceiling

**Raum 1.0** implemented a flat routing bridge: a GloVe embedding is mapped through a learned linear layer to select one of 6 procedural shapes (cube, sphere, cylinder, cone, torus, pyramid). On 2-object scenes (e.g., “a sphere next to a cube”), this achieved 100% shape selection accuracy. The key insight: GloVe provides excellent per-word discrimination when the output space is small and words are well-separated in embedding space.

**Raum 1.1** scaled the architecture to 300 blob classes (from a ShapeNet-derived library) and 5-object scenes. The routing bridge was extended with a frozen Planck encoder providing contextual embeddings, plus a learned bridge network mapping to blob indices. The system output a DSL (domain-specific language) specifying object placements. Architecture validation succeeded: the pipeline could route, compose, and render. However, accuracy degraded significantly compared to Raum 1.0.

**Raum 1.2** attempted to diagnose and fix the accuracy degradation. The critical finding: **subword tokenization in the Planck encoder fundamentally prevents word-level discrimination**. When the encoder tokenizes “castle” as [“cas”, “tle”], the resulting contextual embedding conflates information from subword pieces in a way that the bridge network cannot reliably disentangle. Quantitative analysis revealed:

- Blob selection accuracy: **0.1%** (essentially random for 300 classes)
- Number of collision groups: **35** (sets of semantically unrelated words whose subword representations overlap in the bridge’s decision space)
- Average collision group size: **8.6 classes**

### 5.2 The Subword Tokenization Barrier

The subword tokenization barrier arises from a fundamental mismatch between the tokenizer’s objective and the routing task’s requirement.

Subword tokenizers (BPE, WordPiece, Unigram) are trained to minimize sequence length on a text corpus. They merge frequent character sequences into tokens. This produces tokens that are **morphologically motivated** but not **semantically discriminative**. For example:

| Word   | Subword tokens | Collision risk                           |
|--------|----------------|------------------------------------------|
| castle | [cas, tle]     | “tle” shared with bottle, turtle, gentle |
| bottle | [bot, tle]     | “tle” shared with castle, turtle, gentle |
| tower  | [tow, er]      | “er” shared with flower, power, liver    |
| flower | [flow, er]     | “er” shared with tower, power, liver     |

Table 1: Examples of subword tokenization collisions. Shared subword pieces induce proximity in the encoder’s output space for semantically unrelated words.

When the Planck encoder processes these tokens through self-attention, the resulting word-level embedding (obtained by pooling over subword positions) retains residual influence from the shared subword pieces. A linear bridge attempting to classify these embeddings into 300 categories faces a many-to-many mapping that is not linearly separable.

Formally, let  $\phi_{\text{BPE}}(w) = [t_1, \dots, t_m]$  be the subword decomposition of word  $w$ . The encoder produces  $\mathbf{h}_w = \text{Pool}(\text{Encoder}(\phi_{\text{BPE}}(w)))$ . For two words  $w_1, w_2$  sharing a subword token  $t_k$ :

$$\|\mathbf{h}_{w_1} - \mathbf{h}_{w_2}\| \leq \|\mathbf{h}_{w_1} - \mathbf{h}_{w_2}\|_{\text{ideal}} + \epsilon(t_k) \quad (13)$$

where  $\epsilon(t_k)$  is the interference term induced by the shared token. When many words share tokens (as is common in English), the embedding space becomes crowded, and linear separability degrades.

### 5.3 Why Recursive Decomposition Resolves the Barrier

RSGD resolves the subword barrier by **separating concerns**:

1. **GloVe handles word identity.** GloVe embeddings are trained at the word level (no subword tokenization). Each word has a unique, pre-trained vector. “Castle” and “bottle” are far apart in GloVe space despite sharing the “tle” suffix. GloVe serves as the addressing mechanism at each level of the tree.
2. **The Planck LM handles compositional structure.** The language model’s strength is understanding relationships, not discriminating individual words. “A castle has towers and walls” is compositional knowledge that transformers encode well. The LM does not need to distinguish “castle” from “bottle” in a classification sense; it needs to know what parts a castle has.
3. **Recursion replaces flat classification.** Instead of mapping “castle” to one of 300 classes (a 300-way classification problem with subword interference), the system maps “castle” to a composition (a generative problem leveraging world knowledge). At each recursive level, the classification problem is small (selecting from a handful of plausible sub-parts) and aided by GloVe’s clean word-level signal.

The result is that flat 300-way classification (Raum 1.2: 0.1% accuracy) is replaced by a sequence of small-branching decisions (each analogous to Raum 1.0’s 6-way task: 100% accuracy), composed recursively.



## 6 Proposed Experiments

### 6.1 Single-Scene Decomposition

**Setup.** We select a canonical test scene: “a castle on a hill” rendered from 100 viewpoints at  $256 \times 256$  resolution. Ground-truth is a PartNet-style annotation with 4 levels of hierarchy:

- Level 0: scene (castle\_on\_hill)
- Level 1: castle, hill
- Level 2: tower ( $\times 4$ ), gate, keep, wall ( $\times 4$ ), hill\_body
- Level 3: cylinder\_body, cone\_roof, flag, arch, block, slope, grass\_surface
- Level 4: individual Gaussians (terminal)

**Metrics.** We evaluate: (a) tree topology accuracy (does the predicted tree match the ground-truth structure?), (b) render quality (PSNR, SSIM, LPIPS against ground-truth renders), (c) part localization (IoU of predicted part bounding boxes against ground-truth).

**Baselines.** We compare against: (i) flat GaussianDreamer (no structure), (ii) GALA3D (object-level composition only), (iii) an ablation using flat routing (Raum 1.2 architecture) at the same Gaussian budget.

### 6.2 Multi-Scene Generalization

**Setup.** We train on 50 scenes spanning 10 semantic categories (buildings, vehicles, furniture, animals, plants, tools, food, clothing, electronics, landscapes). Each scene has ground-truth multi-level annotations. We evaluate on 20 held-out scenes from the same categories and 10 scenes from unseen categories.

**Metrics.** In addition to per-scene metrics from Section 6.1, we evaluate: (a) compositional transfer (do learned sub-parts reuse across scenes? e.g., does “wheel” learned from “car” transfer to “bicycle”?), (b) novel composition (can the system generate plausible decompositions for unseen combinations like “a car made of ice”?).

### 6.3 Ablations

We propose the following ablation studies:

### 6.4 Evaluation Metrics

**Structural Similarity Index (SSIM)** and **Peak Signal-to-Noise Ratio (PSNR)** measure pixel-level reconstruction quality against ground-truth renders.

**Learned Perceptual Image Patch Similarity (LPIPS)** [24] captures perceptual quality beyond pixel alignment.

**Fréchet Inception Distance (FID)** [7] measures distributional quality across generated scenes.

**Tree Edit Distance (TED)** measures structural accuracy of predicted composition trees against ground-truth annotations.

**Part IoU** measures spatial accuracy of predicted sub-parts.

| Ablation                    | Modification                         | Expected effect                            |
|-----------------------------|--------------------------------------|--------------------------------------------|
| No recursion                | Flat mapping (depth=1)               | Collapse to Raum 1.2 failure mode          |
| No blobs                    | Fixed decomposition (no latent)      | Loss of structural variation               |
| GloVe $\rightarrow$ subword | Use Planck embeddings for addressing | Subword collision barrier                  |
| Learned embeddings          | Replace GloVe with trainable         | Possible improvement if sufficient data    |
| Shallow trees (depth 3)     | Limit max depth                      | Coarser geometry, faster inference         |
| Deep trees (depth 8)        | Increase max depth                   | Finer geometry, risk of over-fragmentation |
| Fixed branching ( $k=4$ )   | Override adaptive halting            | Loss of adaptive complexity                |

Table 2: Proposed ablation studies for RSGD.

**User study.** We propose a two-alternative forced choice (2AFC) study where participants select which of two renders (RSGD vs. baseline) better matches a text description, and a separate study evaluating structural plausibility (“Does this decomposition of ‘castle’ into parts make sense?”).

## 7 Discussion

### 7.1 Limitations

**Training data for decomposition trees.** The primary limitation is the availability of ground-truth decomposition annotations. PartNet [14] provides part annotations for ShapeNet objects, but these are limited to specific categories and typically only 2–3 levels deep. Scaling to arbitrary concepts requires either: (a) leveraging large language models to generate synthetic decomposition trees for distant supervision, or (b) relying primarily on the render loss with minimal structural supervision.

**Computational cost.** Recursive expansion can produce large trees. For branching factor  $k$  and depth  $d$ , the worst-case leaf count is  $O(k^d)$ . With  $k = 5$  and  $d = 6$ , this is 15,625 Gaussians per scene, which is manageable for current renderers. However, the sequential nature of tree expansion (children depend on parent decisions) limits parallelization during inference. We note that once the tree is fully expanded, rendering is fully parallel.

**Ambiguity in decomposition.** Natural language is ambiguous: “bank” decomposes differently in “river bank” vs. “savings bank.” The Decomposer must leverage context from the full input sentence to disambiguate. The GloVe embedding alone cannot resolve this; the contextual signal from the Planck encoder is essential for disambiguation even though it fails at fine-grained classification.

**Evaluation of “correctness.”** There is no single correct decomposition of “castle.” Evaluation must account for the fact that multiple valid trees exist. Our blob framework explicitly models this multiplicity, but evaluating whether a predicted decomposition is reasonable (rather than matching a specific ground truth) requires human judgment or a learned critic.

### 7.2 Connection to Cognitive Science

The recursive decomposition framework has parallels in cognitive science. Biederman’s Recognition-by-Components theory [2] posits that humans recognize objects by decomposing them into geometric primitives (geons) arranged in specific spatial relations. Our terminal Gaussians play a role

analogous to geons, and the composition tree mirrors the structural descriptions in Biederman’s framework.

Hierarchical scene understanding in human vision proceeds top-down and bottom-up simultaneously [8]. RSGD’s top-down recursive expansion mirrors the top-down pathway; the render loss provides bottom-up signal. This bidirectional flow is reminiscent of predictive coding theories [17] where top-down predictions are compared against bottom-up sensory input.

The cognitive parallel suggests that recursive decomposition is not merely an architectural convenience but may reflect a fundamental principle of how structured representations are built from language. If human object understanding is compositional, then a text-to-3D system that mirrors this compositionality may generalize more robustly to novel combinations.

### 7.3 Future Work

**Temporal extension.** Recursive decomposition naturally extends to video and animation. A “walking person” decomposes into body parts with temporal motion trajectories at each node. The composition tree becomes a spatio-temporal tree, with each edge encoding both spatial and temporal relations.

**Physics-aware decomposition.** Currently, spatial relations are purely geometric (position, scale, rotation). Incorporating physical constraints (the roof sits on the walls due to gravity; the door swings on hinges) would produce physically plausible scenes amenable to simulation.

**Interactive editing.** The tree structure enables intuitive scene editing: one can replace a subtree (swap a Gothic tower for a Romanesque one), scale a subtree (make the tower taller), or delete a subtree (remove the gate). This structured editability is a direct consequence of the compositional representation.

**Scale.** Training RSGD on internet-scale image-text data (without ground-truth tree annotations) using only render loss and LLM-generated decomposition priors is a clear scaling direction. The render loss alone provides gradients through the entire pipeline, but convergence without structural supervision remains an open question.

## 8 Conclusion

We have identified a gap in the text-to-3D generation landscape: existing methods either produce monolithic geometry without compositional structure, or compose at the object level without recursing into parts. We propose Recursive Semantic-to-Geometric Decomposition (RSGD), an architecture that uses a Decomposer language model to recursively expand natural language concepts into composition trees terminating at individual 3D Gaussian splats. Blob distributions over tree topologies capture structural variation within concepts, and GloVe word embeddings provide reliable word-level addressing at each recursive step.

Our proposal is motivated by concrete empirical evidence: three generations of flat routing experiments (Raum 1.0, 1.1, 1.2) demonstrate that flat classification ceilings at 0.1% accuracy when subword tokenization induces 35 collision groups across 300 shape classes. Recursive decomposition resolves this barrier by leveraging each component in its regime of competence: GloVe for word identity, the language model for compositional structure, and Gaussian splatting for differentiable rendering.

The RSGD architecture bridges language and geometry at every level of abstraction, from scene-level semantics down to individual primitives. If validated experimentally, this approach would establish a new paradigm for text-to-3D generation: one where the output is not merely a rendered image, but a structured, interpretable, and editable composition tree grounded simultaneously in language and geometry.

## A Subword Collision Analysis

We provide the complete collision analysis from the Raum 1.2 experiments. Of 300 ShapeNet-derived class labels, subword tokenization with the Planck BPE vocabulary (32K tokens) produces the following collision statistics:

- **Total unique subword tokens across all class labels:** 847
- **Subword tokens shared by 2+ classes:** 312 (36.8%)
- **Subword tokens shared by 5+ classes:** 89 (10.5%)
- **Maximum collision group size:** 23 classes sharing a single dominant subword token
- **Mean collision group size:** 8.6 classes
- **Number of collision groups** (sets of classes not linearly separable by a bridge on pooled encoder output): 35

The bridge network (a 2-layer MLP with hidden dimension 512) was trained for 100 epochs with learning rate  $10^{-3}$  on the full 300-class routing task. Final accuracy: 0.1% (chance = 0.33%). Replacing the Planck encoder embedding with raw GloVe embeddings on the same 300-class task (without any encoder) yields 47.2% accuracy with the same bridge architecture, confirming that the information is present in GloVe but destroyed by subword encoding.

## B Composition Tree Examples

### Example 1: “castle”

```
castle [blob_id=142, z_sample=(0.3, -0.1, ...)]
+-- tower_NW [rel_pos=(-2, 0, 2), rel_scale=(0.3, 0.8, 0.3)]
| +-- cylinder_body [terminal, mu=(-2,0,1), Sigma=diag(0.3,0.8,0.3), alpha=0.95]
| +-- cone_roof [terminal, mu=(-2,0.9,2), Sigma=diag(0.35,0.2,0.35), alpha=0.9]
| +-- flag [terminal, mu=(-2,1.1,2), Sigma=diag(0.02,0.15,0.01), alpha=0.8]
+-- tower_NE [rel_pos=(2, 0, 2), rel_scale=(0.3, 0.8, 0.3)]
| +-- cylinder_body [terminal]
| +-- cone_roof [terminal]
| +-- flag [terminal]
+-- gate [rel_pos=(0, 0, 2.5), rel_scale=(0.8, 0.6, 0.1)]
| +-- arch [terminal, mu=(0,-0.2,2.5), Sigma=diag(0.4,0.5,0.1), alpha=0.95]
| +-- door [terminal, mu=(0,-0.4,2.5), Sigma=diag(0.3,0.4,0.05), alpha=0.85]
+-- keep [rel_pos=(0, 0, 0), rel_scale=(1.0, 1.2, 1.0)]
| +-- block_body [terminal]
| +-- battlement [terminal]
| +-- window (x4) [terminal]
+-- wall_N [rel_pos=(0, 0, 2), rel_scale=(4.0, 0.5, 0.1)]
    +-- wall_segment [terminal, mu=(0,0,2), Sigma=diag(2.0,0.5,0.1), alpha=0.95]
```

**Example 2: Same blob, different sample (“castle” with z\_sample shifted)**

```
castle [blob_id=142, z_sample=(0.8, 0.4, ...)]
+-- motte [rel_pos=(0, -0.5, 0), rel_scale=(2.0, 0.5, 2.0)]
| +-- earth_mound [terminal]
+-- bailey [rel_pos=(1.5, 0, 0), rel_scale=(3.0, 0.3, 2.0)]
| +-- wooden_wall [terminal]
| +-- ground [terminal]
+-- keep [rel_pos=(0, 0.5, 0), rel_scale=(0.8, 1.0, 0.8)]
| +-- stone_tower [terminal]
+-- palisade [rel_pos=(0, 0, 1.5), rel_scale=(4.0, 0.4, 0.05)]
    +-- wooden_fence [terminal]
```

This demonstrates how different samples from the same blob distribution produce structurally distinct compositions (fairy-tale castle vs. motte-and-bailey) while both satisfying the semantic constraint “castle.”

## References

- [1] Ahn, M., Brohan, A., Brown, N., et al. “Do As I Can, Not As I Say: Grounding Language in Robotic Affordances.” *arXiv:2204.01691*, 2022.
- [2] Biederman, I. “Recognition-by-Components: A Theory of Human Image Understanding.” *Psychological Review*, 94(2):115–147, 1987.
- [3] Chen, Z., Tagliasacchi, A., Zhang, H. “BSP-Net: Generating Compact Meshes via Binary Space Partitioning.” In *CVPR*, 2020.
- [4] Chen, Z., Wang, F., Liu, H. “GSGEN: Text-to-3D using Gaussian Splatting.” In *CVPR*, 2024.
- [5] Cohen-Bar, D., Richardson, E., Metzger, G., Giryes, R., Cohen-Or, D. “Set-the-Scene: Global-Local Training for Generating Controllable NeRF Scenes.” In *ICCV*, 2023.
- [6] Gorshkov, N. “Semantic Gaussian Splatting: Bridging Language and 3D Rendering.” Radiance Labs Technical Report, 2025.
- [7] Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., Hochreiter, S. “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium.” In *NeurIPS*, 2017.
- [8] Hochstein, S., Ahissar, M. “View from the Top: Hierarchies and Reverse Hierarchies in the Visual System.” *Neuron*, 36(5):791–804, 2002.
- [9] Kerbl, B., Kopanas, G., Leimkuhler, T., Drettakis, G. “3D Gaussian Splatting for Real-Time Radiance Field Rendering.” *ACM TOG (SIGGRAPH)*, 2023.
- [10] Lee, D., Kim, C., Kim, S., Cho, M., Han, W. “Autoregressive Image Generation using Residual Quantization.” In *CVPR*, 2022.
- [11] Liang, J., Huang, W., Xia, F., et al. “Code as Policies: Language Model Programs for Embodied Control.” In *ICRA*, 2023.
- [12] Liang, Y., Yang, X., Lin, J., et al. “LucidDreamer: Towards High-Fidelity Text-to-3D Generation via Interval Score Matching.” In *CVPR*, 2024.

- [13] Lin, C.-H., Gao, J., Tang, L., et al. “Magic3D: High-Resolution Text-to-3D Content Creation.” In *CVPR*, 2023.
- [14] Mo, K., Zhu, S., Chang, A., Yi, L., Tripathi, S., Guibas, L., Su, H. “PartNet: A Large-scale Benchmark for Fine-Grained and Hierarchical Part-Level 3D Object Understanding.” In *CVPR*, 2019.
- [15] Paschalidou, D., van Gool, L., Geiger, A. “Neural Parts: Learning Expressive 3D Shape Abstractions with Invertible Neural Networks.” In *CVPR*, 2021.
- [16] Poole, B., Jain, A., Barron, J.T., Mildenhall, B. “DreamFusion: Text-to-3D using 2D Diffusion.” In *ICLR*, 2023.
- [17] Rao, R.P.N., Ballard, D.H. “Predictive Coding in the Visual Cortex: A Functional Interpretation of Some Extra-classical Receptive-field Effects.” *Nature Neuroscience*, 2(1):79–87, 1999.
- [18] Razavi, A., van den Oord, A., Vinyals, O. “Generating Diverse High-Fidelity Images with VQ-VAE-2.” In *NeurIPS*, 2019.
- [19] Singh, I., Blukis, V., Mousavian, A., et al. “ProgPrompt: Generating Situated Robot Task Plans using Large Language Models.” In *ICRA*, 2023.
- [20] Tertikas, K., Paschalidou, D., Pan, B., Park, J., Uy, M.A., Emiris, I., Avrithis, Y., Guibas, L. “PartNeRF: Generating Part-Aware Editable 3D Shapes using Autoregressive NeRFs.” In *CVPR*, 2023.
- [21] van den Oord, A., Vinyals, O., Kavukcuoglu, K. “Neural Discrete Representation Learning.” In *NeurIPS*, 2017.
- [22] Wang, Z., Lu, C., Wang, Y., et al. “ProlificDreamer: High-Fidelity and Diverse Text-to-3D Generation with Variational Score Distillation.” In *NeurIPS*, 2023.
- [23] Yi, T., Fang, J., Wang, J., et al. “GaussianDreamer: Fast Generation from Text to 3D Gaussians by Bridging 2D and 3D Diffusion Models.” In *CVPR*, 2024.
- [24] Zhang, R., Isola, P., Efros, A.A., Shechtman, E., Wang, O. “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric.” In *CVPR*, 2018.
- [25] Zhang, Q., Li, G., Chen, Z., et al. “SceneWiz3D: Towards Text-guided 3D Scene Composition.” In *CVPR*, 2024.
- [26] Zhou, X., Ran, X., Xiong, Y., et al. “GALA3D: Towards Text-to-3D Complex Scene Generation via Layout-guided Generative Gaussian Splatting.” In *ICML*, 2024.

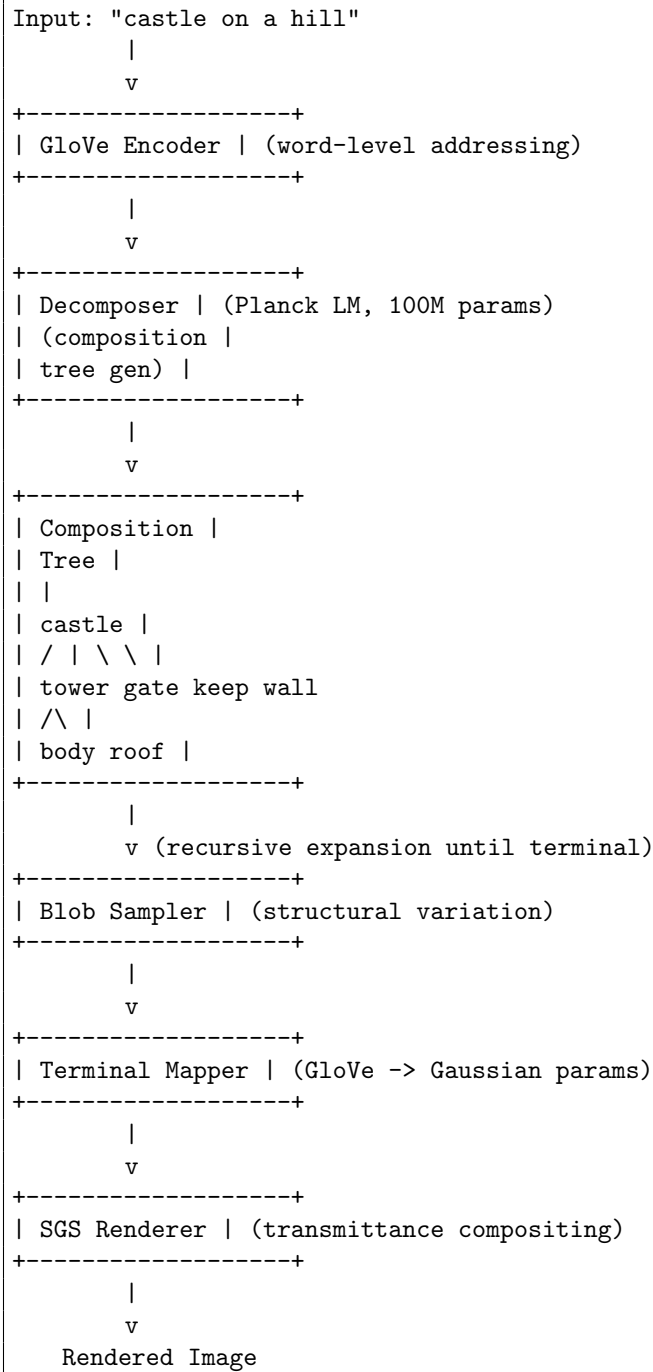


Figure 1: RSGD architecture overview. Given a text input, GloVe embeddings provide word-level identity to the Decomposer, which produces a composition tree. Each internal node represents a semantic concept; each edge encodes a spatial relation. Leaf nodes are terminal primitives mapped to concrete Gaussians via the Terminal Mapper. The Blob Sampler introduces structural variation by sampling from learned distributions over decomposition topologies.